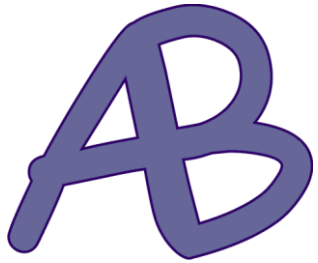
UNIWERSYTET RADOMSKI
im. Kazimierza Pułaskiego

Aplikacje frontendowe

Elementy pływające, pozycjonowanie i display

dr Artur Bartoszewski



Elementy pływające

Dawniej technika ta używana była jako podstawowe narzędzie do tworzenia wielokolumnowych układów stron (layoutów).

Obecnie: pływanie (float) powinno być używane wyłącznie zgodnie ze swoim oryginalnym i węższym przeznaczeniem: **do oblewania elementów prostokątnych tekstem (np. zdjęć w artykułach).**

Ważne: Do budowy układów nowoczesnych stron i portali używa się dziś mechanizmów CSS Flexbox oraz CSS Grid (patrz następne wykłady)

Znaczenie i zachowanie własności float

- ✓ Element, któremu nadano własność `float` zostaje „wypchnięty” jak najdalej w lewo lub w prawo w obrębie swojego kontenera.
- ✓ Naturalny ciąg elementów (strumień) ignoruje go jako blok o pełnej szerokości. W efekcie elementy tekstowe znajdujące się po elemencie pływającym zaczynają go opływać.
- ✓ Elementy znajdujące się za elementem pływającym przesuwają się do góry, układając się obok niego, jeśli jest tam wystarczająco dużo miejsca.

Możliwe wartości:

- `float: left;`
- `float: right;`
- `float: none;` (domyślna w standardzie dokumentu).

Własność **float**

Głównym zastosowaniem własności **float** jest otaczanie obrazów tekstem akapitowym.

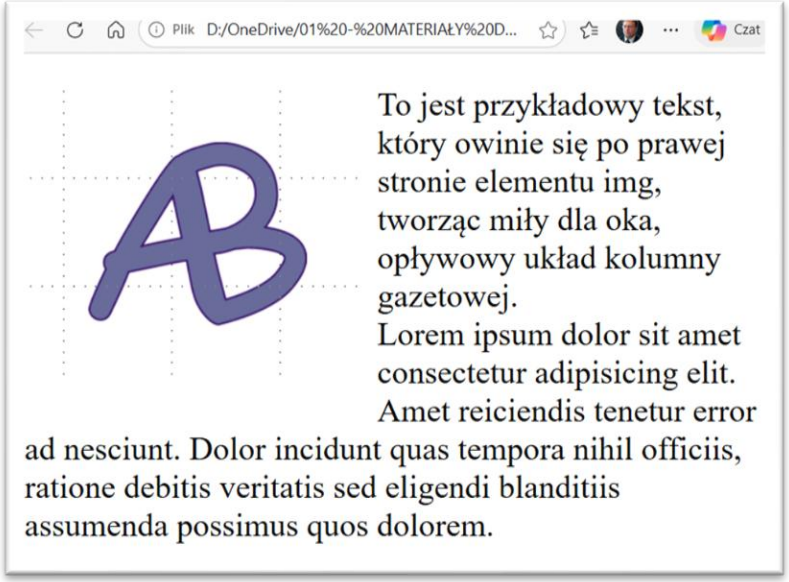
Aby sprawić, by tekst z akapitu płynnie opływał obraz wepchnięty na lewą krawędź strony, przesuwamy obraz w lewo:

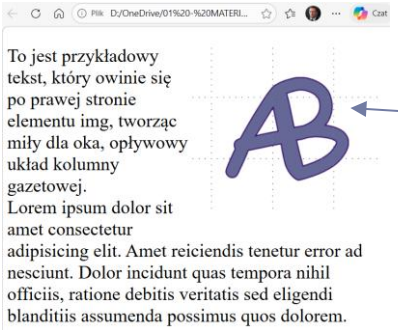
```
.obraz {
    float: left;
}
```

Własność **float**

```
<body>
    <img src="logo.png" alt="Logo AB" class="plywajacy-obraz" />
    <p>To jest przykładowy tekst, ...</p>
</body>
```

```
.plywajacy-obraz {
    float: left;
    margin: 0 15px 15px 0;
    /* Pamiętamy o odsunięciu tekstu od obrazka */
}
```






```
float: left;
```

Własność `clear` - czyszczenie opływania

Często w kodzie po wystąpieniu opływania chcemy, aby w następnym rzędzie elementy zachowywały się już normalnie (przestały opływać obraz i usytuowały się swobodnie pod spodem).

- ✓ Używamy do tego polecenia przerywającego – własności `clear`.
- ✓ Własność ta w zależności od podanej wartości zablokuje element tak, że „przeskoczy” on pływające w jego pobliżu i zatrzyma się w najbezpieczniejszym miejscu pod nimi.

Wartości to:

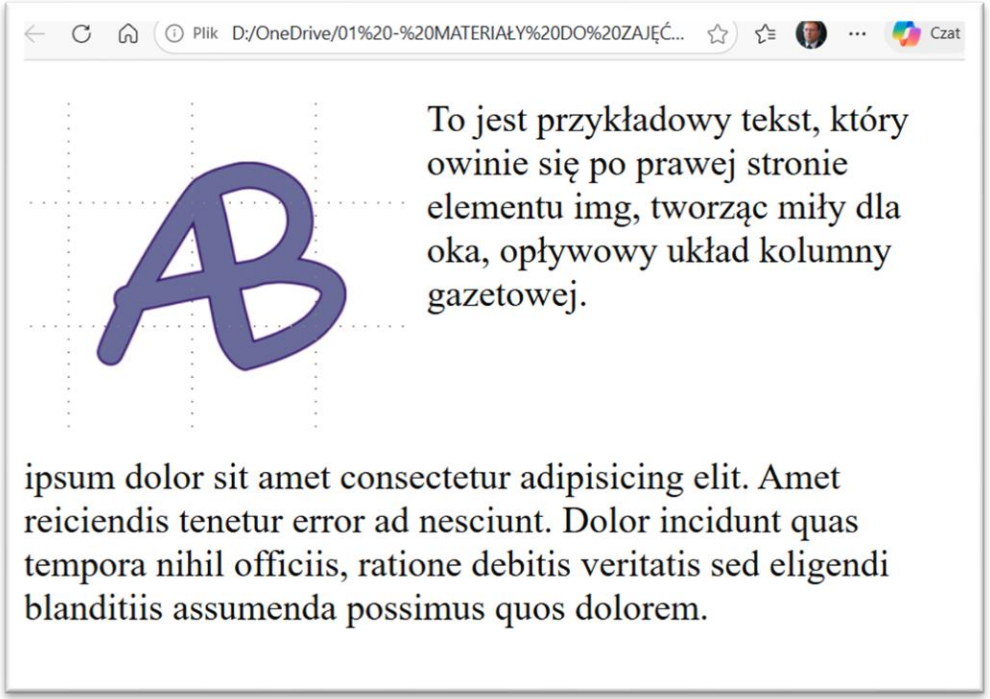
```
✓ clear: left;        /* ignoruje elementy pływające po lewej */
✓ clear: right;       /* ignoruje elementy pływające po prawej */
✓ clear: both;        /* ignoruje wszystkie elementy pływające */
```

Własność clear - czyszczenie opływania

```
<body>
    <img src="logo.png" alt="Logo AB" class="plywajacy-obraz" />
    <p>To jest przykładowy tekst... </p>
    <p class="akapit_pod"> ipsum dolor sit amet...</p>
</body>
```

```
.plywajacy-obraz {
        float: left;
    }

    p.akapit_pod {
        clear: both;
    }
```



Zjawisko „znikającego rodzica”

Poważnym problemem starych układów było zjawisko zapadania wysokości. Element pływający z `float` wydostaje się z wymiarowania pionowego domyślnego rodzica, że na wypadek gdy dany rodzic zawiera wyłącznie elementy float – ten potrafi się „zapaść” do 0 pikseli (brak zarezerwowanej wielkości wysokości).

Oto dwa najbardziej popularne z rozwiązań sprawiających by kontener otulił spowrotem pływające wewnątrz niego elementy:

```
/* dzięki pustemu pseudoelementowi (Standard): */
.kontener::after {
  content: "";
  display: table;
  clear: both;
}
/* dzięki relacji wyznaczającej nowy flow (Najnowszy): */
.kontener {
  display: flow-root;
}
```

Zastosowanie

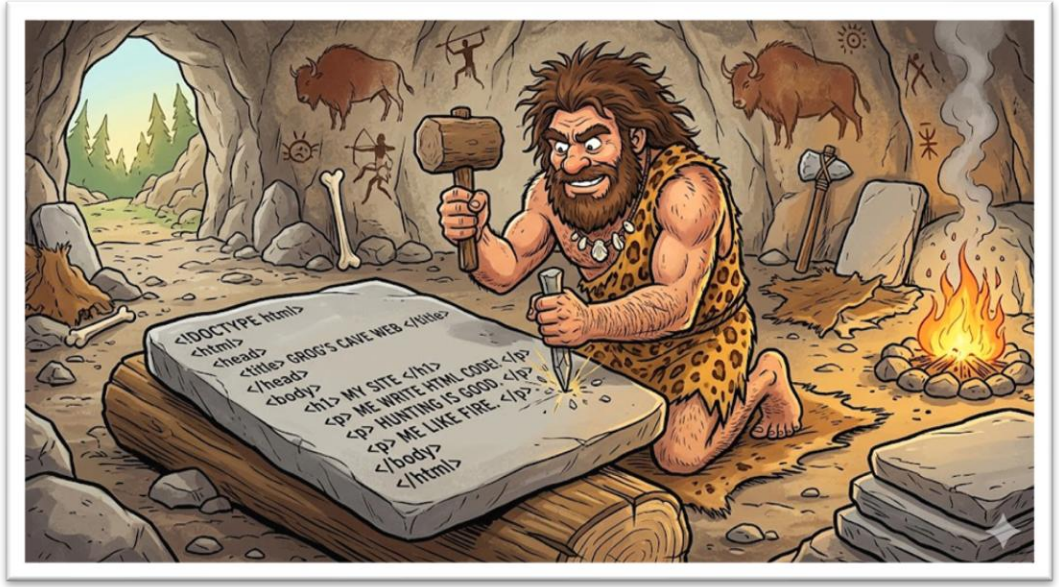
Historycznie float był wykorzystywany do budowy layoutów wielokolumnowych. Współcześnie podejście to uznaje się za przestarzałe ze względu na brak kontroli nad osiami układu oraz problemy z zarządzaniem przepływem.

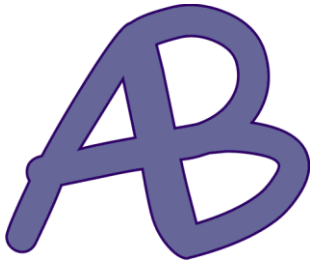
float powinno być używane wyłącznie zgodnie ze swoim oryginalnym i węższym przeznaczeniem: do oblewania elementów prostokątnych tekstem (np. zdjęć w artykułach).

Zalecane alternatywy:

- Flexbox (display: flex) – układ jednowymiarowy
- Grid (display: grid) – układ dwuwymiarowy

Patrz następne wykłady





Pozycjonowanie elementów

Własność position

W CSS własność position pozwala zdefiniować punkt odniesienia, względem którego element ma być pozycjonowany na stronie.

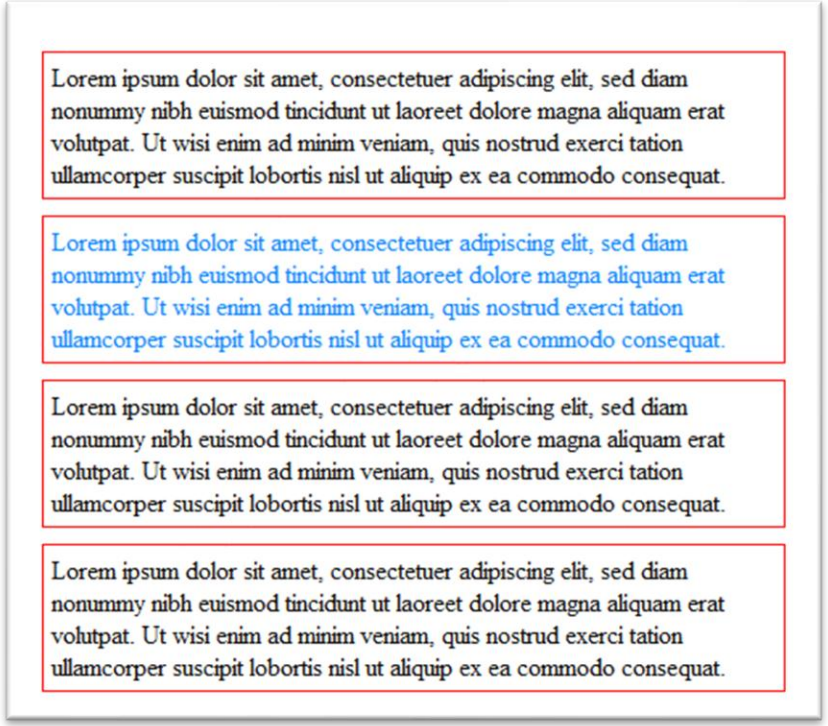
Własność position: może przyjmować jedną z pięciu wartości

1. static (statyczne - nie reaguje) - zachowanie domyślne dla wszystkiego na stronie
2. relative (względne do samego siebie)
3. absolute (całkowicie bezwzględne)
4. fixed (sztywne odnośnie ekranu)
5. sticky (odświeżana relacja lepkości powiązana z przesuwaniem (Scrollowaniem)

Pozycjonowanie statyczne - static

Pozycjonowanie statyczne polega na ułożeniu elementów jeden pod drugim.

- wartość domyślna dla wszystkich elementów,
- element uczestniczy w normalnym przepływie dokumentu (normal flow),
- właściwości pozycjonujące (top, left, right, bottom) są ignorowane,
- brak możliwości użycia z-index.

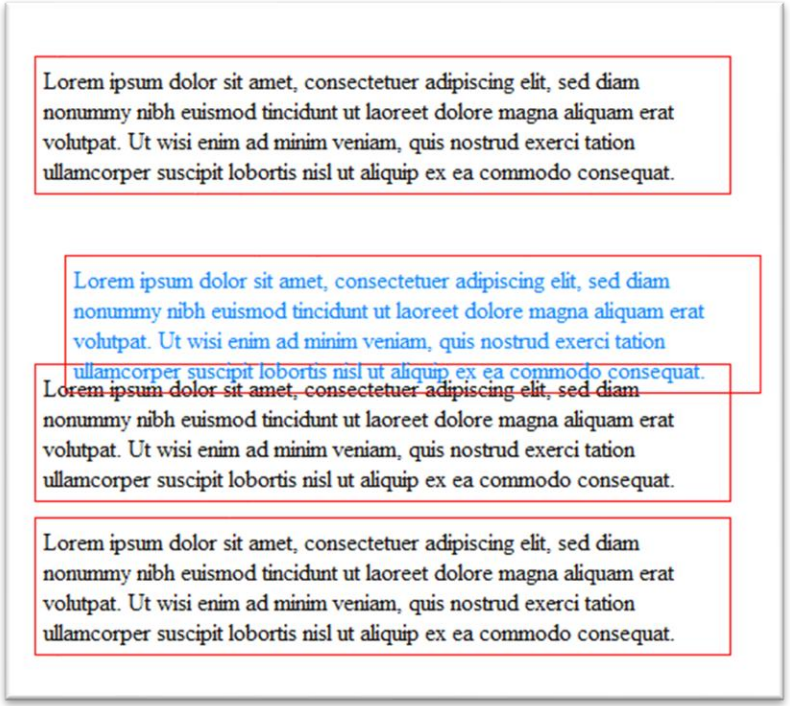


Pozycjonowanie względne - relative

relative: Umożliwia swobodne przesunięcie (poprzez np. `top: 20px`) względem pozycji wyjściowej.

Dzięki temu możemy przesuwać ten element względem jego domyślnego położenia za pomocą własności top, right, bottom i left.

Zazwyczaj wystarczą tylko wartości `top i left.`



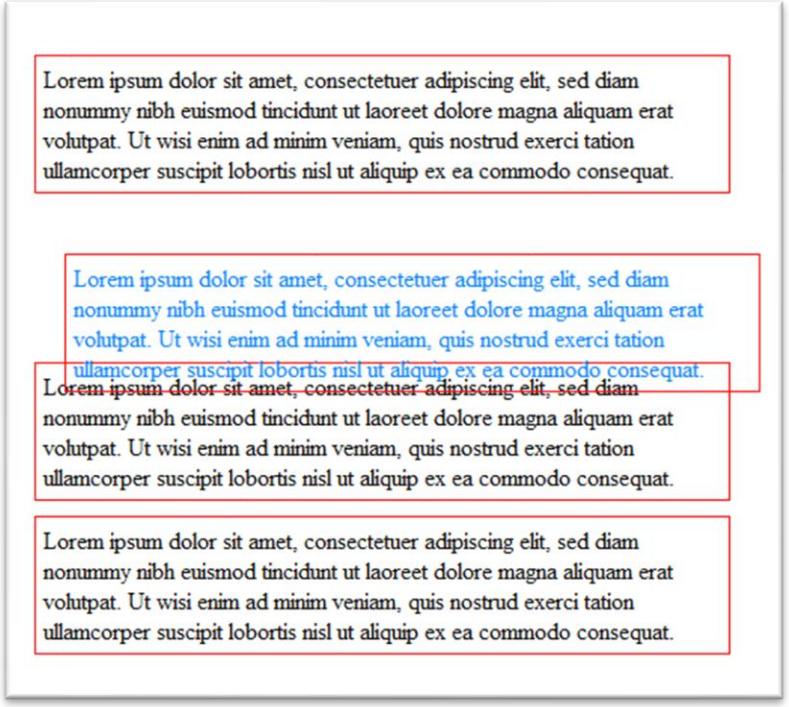
Ustawiamy własność **position** wyróżnionego kolorem akapitu na wartość **relative**.

```
position:relative;
top:30px;
left:20px;
```

Pozycjonowanie względne

Jak widać, mimo że trzeci akapit został przesunięty, nic więcej na stronie się nie zmieniło.

Przesunięcie powoduje powstanie przerwy, ponieważ miejsce w którym znajdował się element przed modyfikacjami współrzędnych **nigdy nie zwalnia przestrzeni**.
Strona interpretuje tak – jakby element w ukryciu pozostał tam gdzie był zaprogramowany i otoczenie zachowuje się domyślnie.



Należy pamiętać, że przesuwając element w ten sposób, trzeba wcześniej wygospodarować dla niego miejsce.

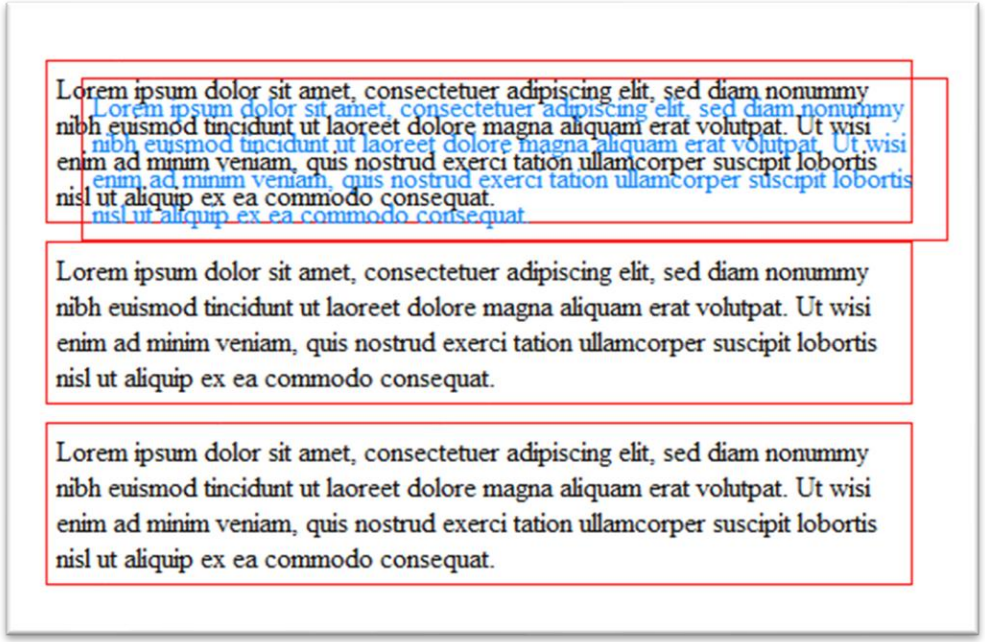
Pozycjonowanie bezwzględne - absolute

Pozycjonowanie bezwzględne pozwala całkowicie wytrącić element z normalnego układu dokumentu.

Nie wpływa na pozycjonowanie innych elementów,

Jak widać na rysunku, miejsce wcześniej zajmowane przez drugi akapit zostało zajęte przez kolejny. Element pozycjonowany bezwzględnie jest całkowicie niezależny od innych otaczających go w kodzie HTML elementów.

Jego położenie jest obliczane względem najbliższego przodka, który ma position inne niż static, w przypadku braku takiego przodka – względem Initial Containing Block (ICB).




```
position:absolute;
top:30px;
left:40px;}
```

Pozycjonowanie bezwzględne

W tym miejscu musimy zapoznać się z pojęciem kontekstu pozycjonowania.

- ✓ Domyślnym kontekstem pozycjonowania elementu pozycjonowanego bezwzględnie jest element Initial Containing Block.
- ✓ Element spozycjonowany bezwzględnie wychodzi poza swój kontener.
- ✓ Jak widać na rysunku z poprzedniego slajdu, własności top i left przesunęły akapit względem ICB, zamiast względem jego domyślnej pozycji w dokumencie.

Kontekst pozycjonowania

P**ozycjonowanie kontekstowe** polega na przesuwaniu danego elementu względem innego za pomocą własności **top, right, left** i **bottom**. Ten drugi element jest właśnie kontekstem pozycjonowania**.**

Kontekstem pozycjonowania jest pierwszy element rodzic który nie posiada własności position: static; Ponieważ jest to domyślna wartość Kontekstem pozycjonowania staje się zwykle element <body>

Kontekstem pozycjonowania może być jednak dowolny element będący przodkiem innego elementu, jeśli jego własności position nada się wartość relative.

Kontekst pozycjonowania

Przyjrzyjmy się poniższemu kodowi:

HTML:

```
<div class="zewnetrzny">
  <div class="wewnetrzny">Lorem ipsum .... </div>
</div>
```

CSS:

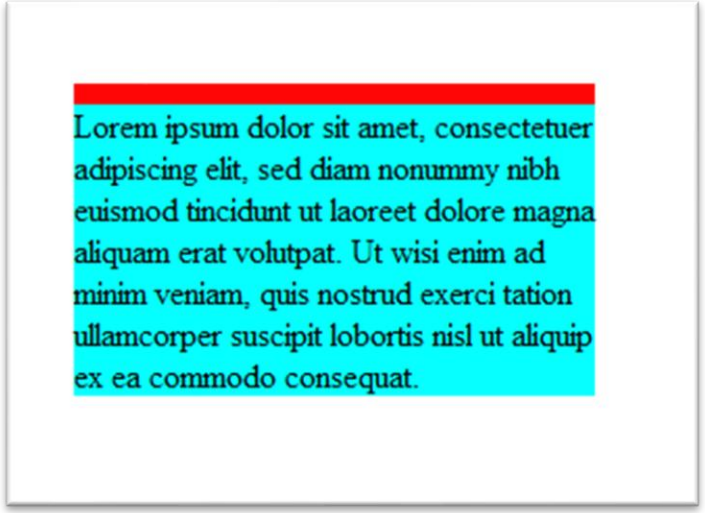
```
.zewnetrzny {
  width:250px;
  margin:50px 40px;
  border-top: 10px solid red;
}
.wewnetrzny {
  top:10px;
  left:20px;
  background:#0ff;
}
```

Kontekst pozycjonowania

- ✓ Nasuwa się pytanie: dlaczego wewnętrzny element div nie jest odsunięty o 10 pikseli w dół i 20 pikseli w lewo względem zewnętrznego elementu div?
- ✓ Zamiast tego górne lewe rogi obu tych elementów znajdują się w tym samym punkcie.

Powodem jest pozycjonowanie statyczne obu elementów. Oznacza to że elementy te wpasowują się w domyślny rozkład elementów na stronie.

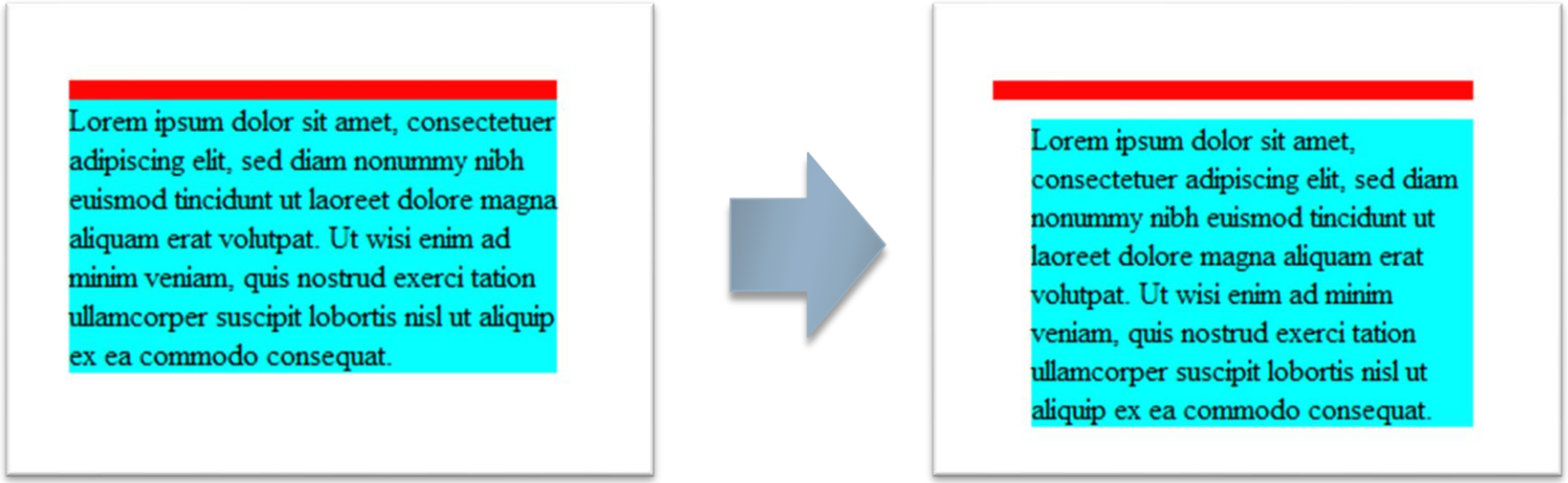
- ✓ Ponieważ zewnętrzny element nie ma żadnej treści, element wewnętrzny zaczyna się w tym samym miejscu.
- ✓ Własności top, right, bottom i left działają tylko jeśli element jest pozycjonowany względnie, bezwzględnie lub w sposób stały



Kontekst pozycjonowania

Aby zewnętrzny kontener div stał się kontekstem pozycjonowania dla kontenera wewnętrznego należy:

1. Ustawić position: absolute dla kontenera wewnętrznego
2. Ustawić position: relative dla kontenera zewnętrznego



Pozycjonowanie stałe - fixed

- ✓ element pozycjonowany względem viewportu,nie reaguje na przewijanie dokumentu,
- ✓ stosowany do elementów UI takich jak przyciski, paski nawigacji, overlaye.

Pozycjonowanie stałe (fixed) jest podobne do bezwzględnego. Różnica polega na tym, że element jest pozycjonowany w odniesieniu do okna przeglądarki (viewportu). Dzięki temu dany element nie przesuwa się w miarę przewijania strony.

Element pozycjonowany w ten sposób nie przesuwa się podczas scrollowania elementu body.

```
.button {
    position: fixed;
    bottom: 20px;
    right: 20px;
}
```

Pozycjonowanie sticky

position : sticky

- element początkowo zachowuje się jak relative,
- po osiągnięciu zdefiniowanego progu (np. top: 0) zmienia zachowanie na zbliżone do fixed,
- zależny od kontenera przewijania (scroll container)

Jest miksem pomiędzy ustawieniem fixed i relative.
Element jest traktowany jako realtive, czyli skroluje razem ze stroną dopóki pewna wartość wysokości strony nie zostanie przekroczona.
Od tego momentu traktowany jest jako element jako pozycjonowany względem viewportu - fixed

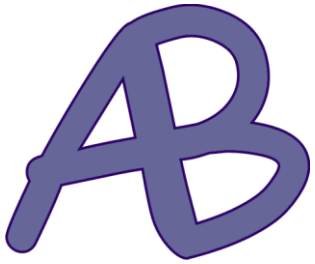
```
.header {
    position: sticky;
    top: 0;
}
```

Na marginesie:

Jak sprawdzić wsparcie dla poszczególnych własności CSS

Chrome	Edge *	Safari	Firefox	Opera	IE	Chrome for Android	Safari on iOS *	Samsung Internet	Opera Mini *	Opera Mobile	Browser for Android	Android Browser *	Firefox for Android	QQ Browser	Baidu Browser	KaiOS Browser
4-22																
[2] 23-36																
37-51	12-15	3.1-6	2-25	10-38			3.2-5.1									
[2] 52-55	[6] 16-18	[5] 6.1-7	[1] 26-31	[2] 39-41			[5] 6-7.1									
[4] 56-90	[4] 79-90	7.1-12.1	[3] 32-58	[4] 42-77			8-12.5	4-5.4								
91-145	91-144	13-26.2	59-147	78-126	6-10		13-26.2	6.2-28		12-12.1		2.1-4.4.4				[3] 2.5
146	145	26.3	148	127	11	145	26.3	29	all	80	15.5	145	147	[4] 14.9	13.52	3.1
147-149		26.4-TP	149-151				26.4									

https://caniuse.com



Display

Model formatowania elementu w drzewie renderowania (render tree), który definiuje jak element jest reprezentowany i jak uczestniczy w układzie (layout) dokumentu.

Kontekst **display**

Właściwość display określa trzy kluczowe aspekty:

a) Typ generowanego box-a

Czy element generuje:

- box blokowy (`block`)
- box liniowy (`inline`)
- box typu `inline-block`
- box elastyczny (`flex`)
- box siatki (`grid`)

b) Uczestnictwo w przepływie dokumentu

Czy element:

- bierze udział w normal flow (`block, inline`)
- tworzy własny kontekst (`flex, grid, flow-root`)
- jest całkowicie usunięty (`display: none`)

c) Kontekst formatowania

display decyduje, jaki algorytm layoutu zostanie użyty:

- Block Formatting Context (BFC) - np. `block, flow-root`
- Inline Formatting Context (IFC) - `inline`
- Flex Formatting Context - `flex`
- Grid Formatting Context - `grid`

Kontekst **display**

Poza własnością position każdy element ma także własność display. Mimo iż własność ta może przyjmować kilka wartości, najczęściej używane są dwie: block i inline (o flex i grid porozmawiamy na następnych wykładach).

Tym, którzy przespali poprzednie wykłady, przypominam różnicę między elementami blokowymi (block) a liniowymi (inline):

- ✓ Elementy blokowe, na przykład akapity, nagłówki czy listy, układają się jeden nad drugim w oknie przeglądarki.
- ✓ Elementy liniowe, na przykład <a> oraz , układają się jeden obok drugiego w oknie przeglądarki. Przechodzą do nowej linii dopiero wówczas, gdy w aktualnej nie ma dla nich wystarczająco dużo miejsca.

Kontekst **display**

Możliwość zamiany elementów blokowych w liniowe i odwrotnie, jak poniżej:

Domyślnie: element blokowy → `p { display: inline; }` → element liniowy

Domyślnie: element liniowy → `a { display: block; }` → element blokowy

Kontekst **display**

`display: block`

- element zajmuje całą dostępną szerokość,
- powoduje przejście do nowej linii,
- umożliwia pełną kontrolę nad wymiarami.

`display: inline`

- element renderowany w linii tekstu,
- brak możliwości ustawienia szerokości i wysokości,
- zależny od kontekstu typograficznego.

`display: inline-block`

- element zachowuje się jak inline w kontekście przepływu,
- umożliwia ustawienie wymiarów jak element blokowy,
- wykorzystywany w komponentach UI (np. przyciski).

Kontekst **display**

Ukrywanie elementów

`display: none`

- ✓ Powoduje ona, że element i wszystkie zagnieżdżone w nim elementy stają się niewidoczne na stronie.
- ✓ Miejsce normalnie zajmowane przez ten element nie jest wtedy przez niego zajmowane.

Wygląda to tak, jakby kod HTML tego elementu w ogóle nie istniał

`visibility: hidden`

- ✓ Element pozostaje w drzewie renderowania, jednak nie jest widoczny,
- ✓ zachowuje swoją przestrzeń w układzie.

visibility: hidden powoduje, że element jest niewidoczny, ale nadal zajmuje przeznaczone dla niego miejsce).

Literatura:

W prezentacji wykorzystano fragmenty i przykłady z książki:

- Wyke-Smitch Ch.; *CSS Witryny szyte na miarę*. Helion